Advanced Programming
Python

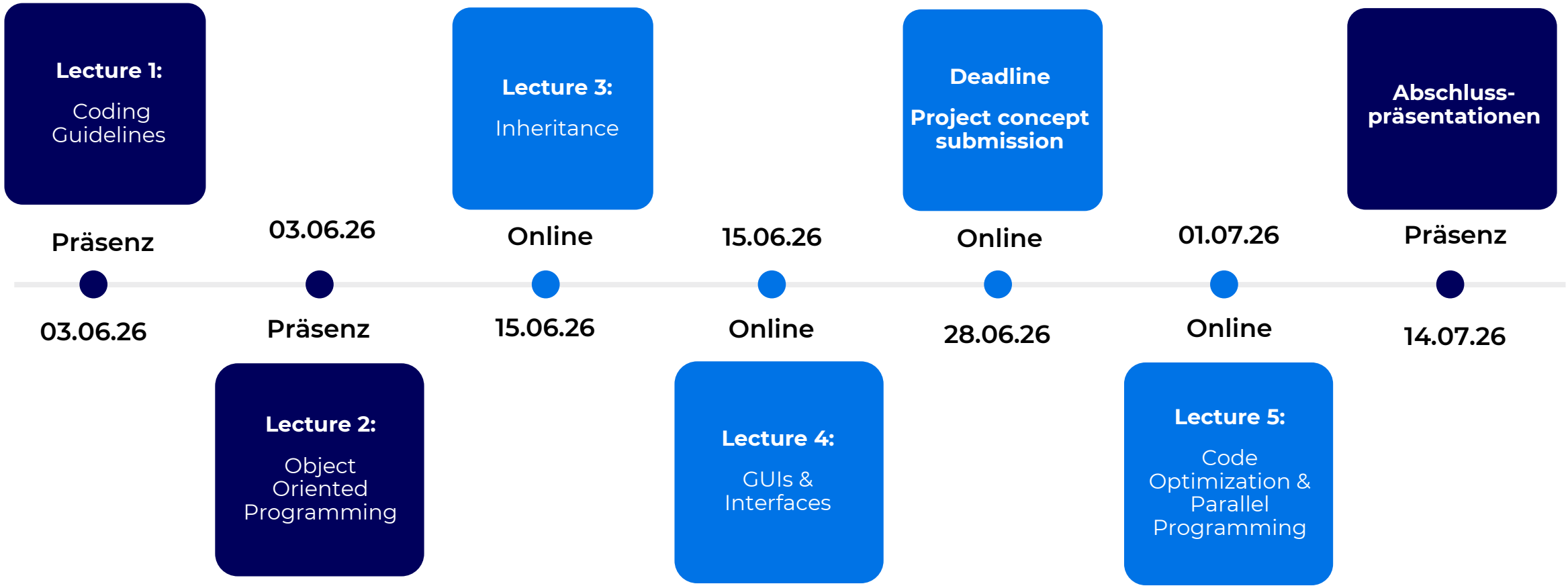
01.07.2026

Marius Kiskemper
Marius.Kiskemper@atos.ai

Atos

Organisatorisches

Termine



Recap

Recap

Abstract Base Classes

- Once a parent class defines abstractmethods it can not instantiate an object anymore
- Using abstractmethods makes the parent class „abstract“ (only used for guiding the implementation)
- In most cases the abstractmethod has no content

```python
from abc import ABC, abstractmethod


class Person(ABC):
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def __str__(self):
        return f"This person is called {self.firstname} {self.lastname}"

    @abstractmethod
    def print_object_info(self):
        pass

    @abstractmethod
    def print_type(self):
        pass


person_1 = Person("John", "Doe")
```

```
    person_1 = Person("John", "Doe")
               ^^^^^^^^^^^^^^^^^^^^^
TypeError: Can't instantiate abstract class Person with abstract methods print_object_info, print_type
```

Recap

Abstract Base Classes

```python
class Student(Person):

    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def print_object_info(self):
        print("My attributes are:")
        for attr in dir(self):
            if not attr.startswith('__') and not callable(getattr(self, attr)):
                print(attr)


student_1 = Student("Mary", "Sue", 2019)
student_1.print_object_info()
```

```
    student_1 = Student("Mary", "Sue", 2019)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: Can't instantiate abstract class Student with abstract method print_type
```

```python
class Student(Person):

    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def print_object_info(self):
        print("My attributes are:")
        for attr in dir(self):
            if not attr.startswith('__') and not callable(getattr(self, attr)):
                print(attr)

    def print_type(self):
        print(f"The type of this object is {self.__class__.__name__}")


student_1 = Student("Mary", "Sue", 2019)
print(student_1)
print("---")
student_1.print_object_info()
print("---")
student_1.print_type()
```

```
This person is called Mary Sue
---
My attributes are:
_abc_impl
firstname
graduation_year
lastname
---
The type of this object is Student
```


Atos

Recap

Abstract Base Classes

- It is possible to execute the abstractmethods using super()
- Useful when something should be repeated in every subclass implementation of this method
- Useful if having multi level inheritance, where not each level must overwrite

```python
class Person(ABC):

    @abstractmethod
    def print_type(self):
        print("Printing the class name:")
```

```python
class Student(Person):

    def print_type(self):
        super().print_type()
        print(f"The type of this object is {self.__class__.__name__}")


student_1 = Student("Mary", "Sue", 2019)
student_1.print_type()
```

```
Printing the class name:
The type of this object is Student
```


Atos

Recap

Abstract Base Classes

- When the abstractmethods are overriden they behave like every other method
- Advantage is that during defining the parent class you can make sure that these methods will be implemented in the subclass
- Access to the remaining methhods stays the same

```python
class Student(Person):

    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def print_object_info(self):
        print("My attributes are:")
        for attr in dir(self):
            if not attr.startswith('__') and not callable(getattr(self, attr)):
                print(attr)

    def print_type(self):
        print(f"The type of this object is {self.__class__.__name__}")


student_1 = Student("Mary", "Sue", 2019)
print(student_1)
print("---")
student_1.print_object_info()
print("---")
student_1.print_type()
```

```
This person is called Mary Sue
---
My attributes are:
_abc_impl
firstname
graduation_year
lastname
---
The type of this object is Student
```

Recap

Multi-Level Inheritance

- Multiple classes inherit in order
- Super() always goes to the next higher layer

```python
class Person():
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def __str__(self):
        return f"This person is called {self.firstname} {self.lastname}"

    def print_name(self):
        print(f"Name of person is {self.firstname, self.lastname}")


class Student(Person):
    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def print_name(self):
        print(f"Name of student is {self.firstname} {self.lastname}")

    def print_year(self):
        print("Graduation year is ", self.graduation_year)


class Programmer(Student):
    def write_code(self, exercise):
        print("Writing code to solve the exercise ", exercise)

    def super_test(self):
        super().print_name()


programmer_1 = Programmer("Progra", "Mierer", 2020)    "Progra": Unknown word.
print(programmer_1)
programmer_1.print_year()
programmer_1.write_code("Implement multi-level inheritance")
programmer_1.super_test()

```

```
This person is called Progra Mierer
Graduation year is  2020
Writing code to solve the exercise  Implement multi-level inheritance
Name of student is Progra Mierer
```

Addition

Multi-Inheritance

- A class can inherit from two other classes
- Super() goes trough the parent classes from left to right an looks for the method name

```python
class A():
    def get_message(self):
        print("Class A")


class B():
    def get_message(self):
        print("Class B")


class C(A, B):
    def result(self):
        super().get_message()


instance = C()
instance.result()
```

Addition

Multi-Inheritance

- To determine the inheritance order, Python gives the Method Resolution Order (MRO)
  - Determined by C3 linearization algorithm

```python
# Method Resolution Order (MRO) (with C3 linearization algorithm)

class Person:
    def greet(self):
        print("Hello from Person")

class Student():
    def greet(self):
        print("Hello from Student")

class Programmer(Person, Student):
    def greet(self):
        super().greet()

programmer_1 = Programmer()
programmer_1.greet()

print(programmer_1.__class__.__mro__)
print(Programmer.__mro__)
print(Programmer.mro())
```

```
Hello from Person
(<class '__main__.Programmer'>, <class '__main__.Person'>, <class '__main__.Student'>, <class 'object'>)
(<class '__main__.Programmer'>, <class '__main__.Person'>, <class '__main__.Student'>, <class 'object'>)
[<class '__main__.Programmer'>, <class '__main__.Person'>, <class '__main__.Student'>, <class 'object'>]
```

01

General Definition – Parallel Programming

What is Parallel Programming?

Definition

- Running **multiple parts of a program simultaneously**
- Possible because of multiple CPUs (Central Processing Units)
- → Independent cores that can run seperate instructions
- Dividing the work into seperate logical packages that run in parallel

- Why?

→ Performance of language Python per core itself plateaued → performance gains now come from more cores

→ A normal Python program only uses one core → all other idle

What is Parallel Programming?

Benefits

- 1 process equals 1 logical package (1 part in single responsiblity)
- Taking advantage of multiple cores (parallelization)
- → program runs faster
- → making better use of existing CPU resources
- → program is well structured
- Especially advantageous when used in CPU heavy computations
  - Arithmetical tranformations on large array, vector math, video processing, ML predictions

What is Parallel Programming?

Risks

- Multiple parallel tasks could refer to the same data and cause **huge inconsistencies**
- Parallel Programming can make a project a lot more complex
  - Difficult for other developers to follow
- Hard to implement perfectly (only then parallelization will give a benefit)

What is Parallel Programming?

Realisation

- Concurrency: Dealing with many things at once
- Parallelism: Doing many things at once

- To use multiple CPUs, special constructs are needed
- → Processes and Threads
- These constructs are necessary for the operating system to understand and seperate the tasks

What is Parallel Programming?

Keep in mind

- Two kinds of work:
  - CPU-bound → limited by processor (e.g. math, parsing, compression, ...) → Bottleneck = computation
  - I/O-bound → limited by waiting (Network, database, API calls) → Bottleneck = waiting

  →How to optimize the code depends on which kind of work is done

- Speedup is capped by fraction of code that can be parallelized

What is Parallel Programming ?

Baseline

```python
def cpu_task(n=5_000_000):
    """CPU-bound: pure computation, keeps a core 100% busy."""
    total = 0
    for i in range(n):
        total += i * i
    return total

def io_task(seconds=1):
    """I/O-bound: just waiting (simulates a network/DB call)."""
    time.sleep(seconds)

def timed(label, func, repeats):
    start = time.perf_counter()
    for _ in range(repeats):
        func()
    print(f"{label:<28}: {time.perf_counter() - start:.2f}s")

if __name__ == "__main__":
    timed("4x CPU task (sequential)", cpu_task, 4)
    timed("4x I/O task (sequential)", io_task, 4)
```

```
4x CPU task (sequential)    : 0.97s
4x I/O task (sequential)    : 4.00s
```

02

Processes

What are processes ?

Definition

- Abstractions that **represent a compute task**
- OS assigns own memory to each process
- Processes access only their assigned memory to stay independent and isolated
- Every **process contains one or more threads**
- Process-based parallelism is the most common way of parallelization in programming

What are processes ?

Implementation

Instantiation of the Process class can call defined functions and run seperate / different function calls in parallel

```python
from multiprocessing import Process


def numbers(start_num):
    total = 0
    for i in range(start_num):
        for j in range(start_num**2):
            sum_ij = i+j
            total += sum_ij

    print(total)


if __name__ == '__main__':
    p1 = Process(target=numbers, args=(100,))
    p2 = Process(target=numbers, args=(500,))
    p1.start()
    p2.start()
```

What are threads?

Implementation for I/O tasks

```python
from concurrent.futures import ProcessPoolExecutor

def cpu_task(n=5_000_000):
    total = 0
    for i in range(n):
        total += i * i
    return total

if __name__ == "__main__":
    start = time.perf_counter()
    with ProcessPoolExecutor(max_workers=4) as pool:
        list(pool.map(cpu_task, [5_000_000] * 4))
    print(f"4x CPU with processes: {time.perf_counter() - start:.2f}s")
```

```
4x CPU task (sequential)    : 1.01s
```

```
4x CPU with processes: 1.55s
```

No speedup !

Why? → Each process gets its own interpreter and its own GIL → processes are expensive to start, and data must copied between them

→ Only worth it if computation is heavy

Atos

What are threads?

Implementation for I/O tasks

```python
def cpu_task(n=50_000_000):
    total = 0
    for i in range(n):
        total += i * i
    return total

if __name__ == "__main__":
    start = time.perf_counter()
    with ProcessPoolExecutor(max_workers=4) as pool:
        list(pool.map(cpu_task, [50_000_000] * 4))
    print(f"4x CPU with processes: {time.perf_counter() - start:.2f}s")
```

```
4x CPU task (sequential)    : 9.54s
```

```
4x CPU with processes: 4.23s
```

Speedup, because of heavy computation

03

Threads

What are threads ?

Definition

- Threads are the **subparts of processes**
- They are the atomic compute task and not only an abstract representation
- Every thread inside a process shares the same memory with sibling threads
- Every thread can access all data structures of its parent process
- Threads are in general faster (as they are more lightweight) and more convenient to use

What are threads?

Implementation for I/O tasks

```python
from concurrent.futures import ThreadPoolExecutor

def io_task(seconds=1):
    time.sleep(seconds)

start = time.perf_counter()
with ThreadPoolExecutor(max_workers=4) as pool:
    pool.map(io_task, [1, 1, 1, 1])
print(f"4x I/O with threads: {time.perf_counter() - start:.2f}s")
```

```
4x I/O task (sequential)    : 4.00s
4x I/O with threads: 1.00s
```

ThreadPoolExecutor and ProcessPoolExecutor have the identical interface

→ Learn to use the single interface and choose the right one for the current workload

Atos

04

Global Interpreter Lock

Atos

Processes and Threads

The difference

Global Interpreter Lock (GIL)

- Protects access to storage (e.g. Python objects)
- **Stops multiple threads from accessing the same objects in parallel**
- When one Thread accesses a ressource, the GIL locks all other threads of the process
- Can happen hundreds of times per second → slows down the program massively
- Prevents race conditions
- This problem only occurrs with Threads
  - Because parallel Threads access the same storage
  - Whereas parallel processes are assigned to seperate memory and have an own GIL each

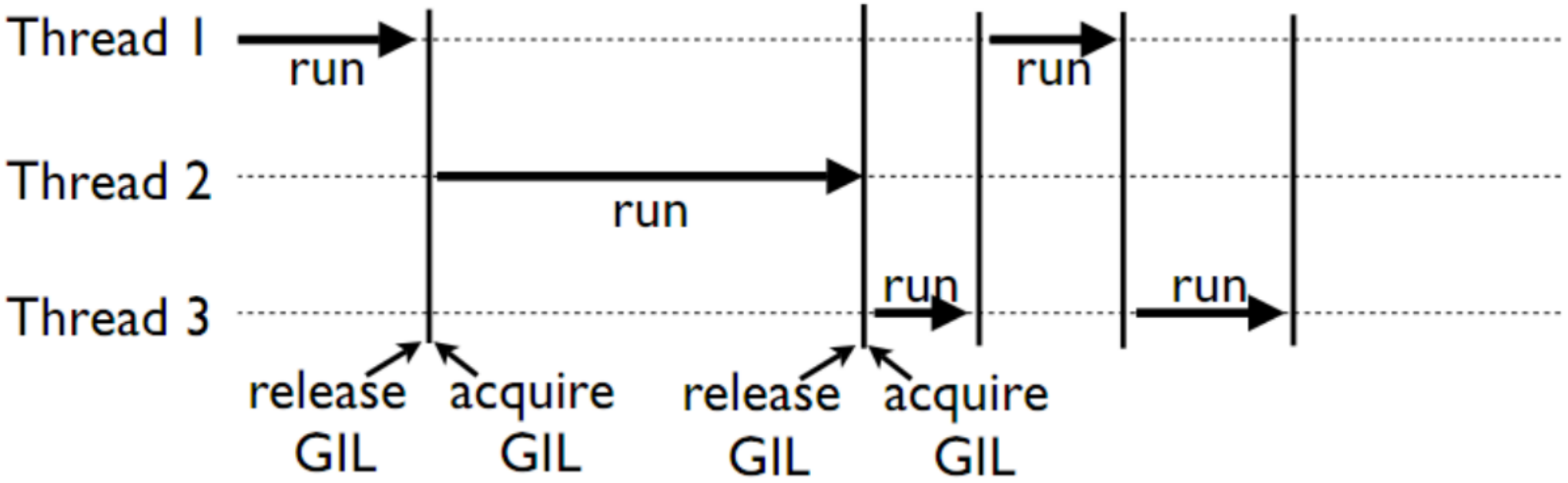
→ Threads are no real parallelization

→ But: GIL released during I/O waits

Processes and Threads

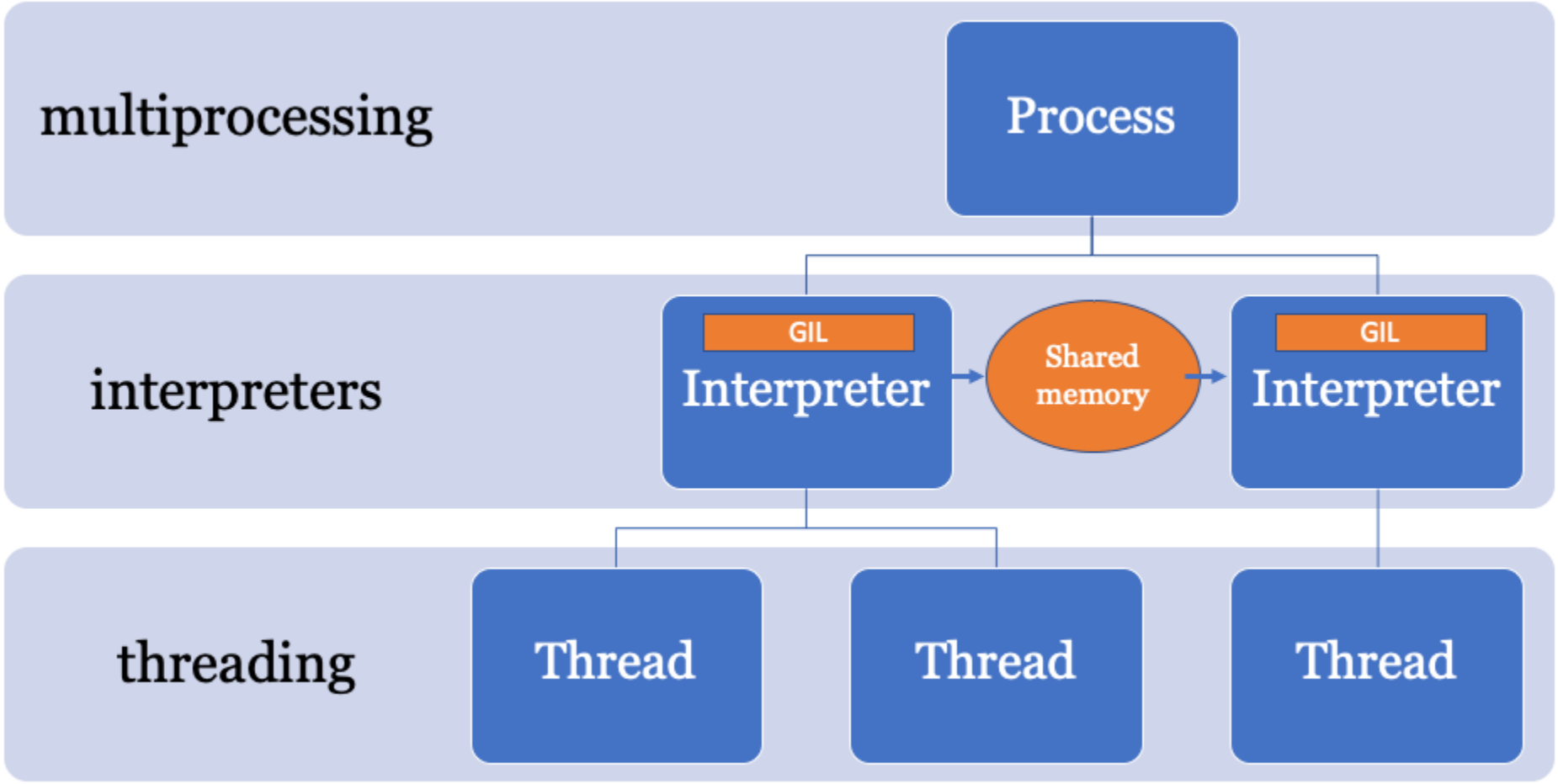
Global Interpreter Lock visualized

No Threads from the same process can run in parallel



Processes and Threads

Global Interpreter Lock visualized



05

Time differences

Measuring parallelization

Comparing execution time

- Measuring parallelization trough **execution time**
- **Time** module to track the execution time
- Timestamp before and after execution
- Difference is needed duration

```python
import threading
import time


def numbers(start_num):
    total = 0
    for i in range(start_num):
        for j in range(start_num**2):
            sum_ij = i+j
            total += sum_ij

    print(total)


if __name__ == '__main__':
    print("Threading:")

    start_time = time.time()

    t1 = threading.Thread(target=numbers, args=(500,))
    t2 = threading.Thread(target=numbers, args=(500,))

    t1.start()
    t2.start()

    t1.join()
    t2.join()

    end_time = time.time()

    duration = end_time-start_time
    print("Duration: ", duration)
```


Atos

Measuring parallelization

Comparing execution time

Multiprocessing

```
Multiprocessing:
15656125000000
15656125000000
Duration:  6.042781591415405
```

- Multiprocessing allows True parallelization
- Process can be parallelized as many times as CPU cores are available
- Becomes more relevant when the workloads are heavier

Threading

```
Threading:
15656125000000
15656125000000
Duration:  12.420454740524292
```

- Threading does not enable parallelization
- Threads are executed sequentially
- Are simply the same as calling the functions in the known way
- Can only be used for real parallelization if the threads belong to separate processes

Measuring parallelization

Comparing CPU usage

- Measuring parallelization trough **CPU usage**
- **psutil** module calculates the CPU allocation at a given moment
- Returns info like available RAM, used RAM, general CPU allocation, ….

```python
import psutil

print("CPU usage:")
print(psutil.cpu_percent())
print(dict(psutil.virtual_memory()._asdict()))
```

```
CPU usage:
31.0
{'total': 34010619904, 'available': 5353287680, 'percent': 84.3, 'used': 28657332224, 'free': 5353287680}
```

Measuring parallelization

Checking the benefit of parallel programming

```python
if __name__ == '__main__':
    print("Multiprocessing:")

    start_time = time.time()

    p1 = Process(target=numbers, args=(500,))
    p2 = Process(target=numbers, args=(500,))
    p3 = Process(target=numbers, args=(500,))
    p4 = Process(target=numbers, args=(500,))
    p5 = Process(target=numbers, args=(500,))
    p6 = Process(target=numbers, args=(500,))
    p7 = Process(target=numbers, args=(500,))
    p8 = Process(target=numbers, args=(500,))

    p1.start()
    p2.start()
    p3.start()
    p4.start()
    p5.start()
    p6.start()
    p7.start()
    p8.start()

    for _ in range(20):
        print("CPU Percent: ", psutil.cpu_percent())
        time.sleep(0.5)
```

```
CPU Percent:  13.3
CPU Percent:  22.3
CPU Percent:  24.6
CPU Percent:  27.7
CPU Percent:  26.8
CPU Percent:  25.5
CPU Percent:  25.9
CPU Percent:  20.6
CPU Percent:  24.5
```

More parallel processes increase CPU usage

```
CPU Percent:  58.7
CPU Percent:  73.6
CPU Percent:  78.6
CPU Percent:  63.9
CPU Percent:  68.8
CPU Percent:  57.2
CPU Percent:  60.3
CPU Percent:  55.5
CPU Percent:  57.6
CPU Percent:  50.6
CPU Percent:  52.8
```

```python
if __name__ == '__main__':
    print("Multiprocessing:")

    start_time = time.time()

    p1 = Process(target=numbers, args=(500,))
    p2 = Process(target=numbers, args=(500,))
    p3 = Process(target=numbers, args=(500,))
    p4 = Process(target=numbers, args=(500,))

    p1.start()
    p2.start()
    p3.start()
    p4.start()

    for _ in range(20):
        print("CPU Percent: ", psutil.cpu_percent())
        time.sleep(0.5)
```


Atos

06

Code optimization

Code optimization

General notes

- Parallelism is not the thing that solves everything
- → Is rather a last optimization point
- A better algorithm usually beats effect of parallelization
- Priorities: Right algorithm → Right data structure → built-ins → parallelism

Code optimization

Good algorithms beat everything

```python
data_list = list(range(1_000_000))
data_set  = set(data_list)
target = 999_999

t_list = timeit.timeit(lambda: target in data_list, number=100)
t_set  = timeit.timeit(lambda: target in data_set,  number=100)
print(f"list lookup (O(n)): {t_list:.4f}s")
print(f"set  lookup (O(1)): {t_set:.6f}s")
```

```
list lookup (O(n)): 0.7086s
set  lookup (O(1)): 0.000008s
```


Atos

Code optimization

Caching

```python
from functools import lru_cache

def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)

@lru_cache(maxsize=None)
def fib_cached(n):
    return n if n < 2 else fib_cached(n-1) + fib_cached(n-2)

print("no cache: ", timeit.timeit(lambda: fib(32), number=1))
print("lru_cache:", timeit.timeit(lambda: fib_cached(32), number=1))
```

```
no cache:  0.2910474000964314
lru_cache: 5.3999945521354675e-05
```


Atos

Code optimization

Caching

```python
import sys

eager = [x*x for x in range(1_000_000)]   # builds the whole list in RAM
lazy  = (x*x for x in range(1_000_000))   # computes on demand

print(sys.getsizeof(eager), "bytes")    "getsizeof": Unknown word.
print(sys.getsizeof(lazy),  "bytes")    "getsizeof": Unknown word.
```

```
8448728 bytes
208 bytes
```

Code optimization

Cheap wins

- Use built-in methods / 3rd party modules (numpy)
- Use comprehensions instead of append
- Use str.join() instead of +=
- Choose the correct and most efficient data structure
- Always measure when optimizing runtime

07

Exercise Time

Exercise Time

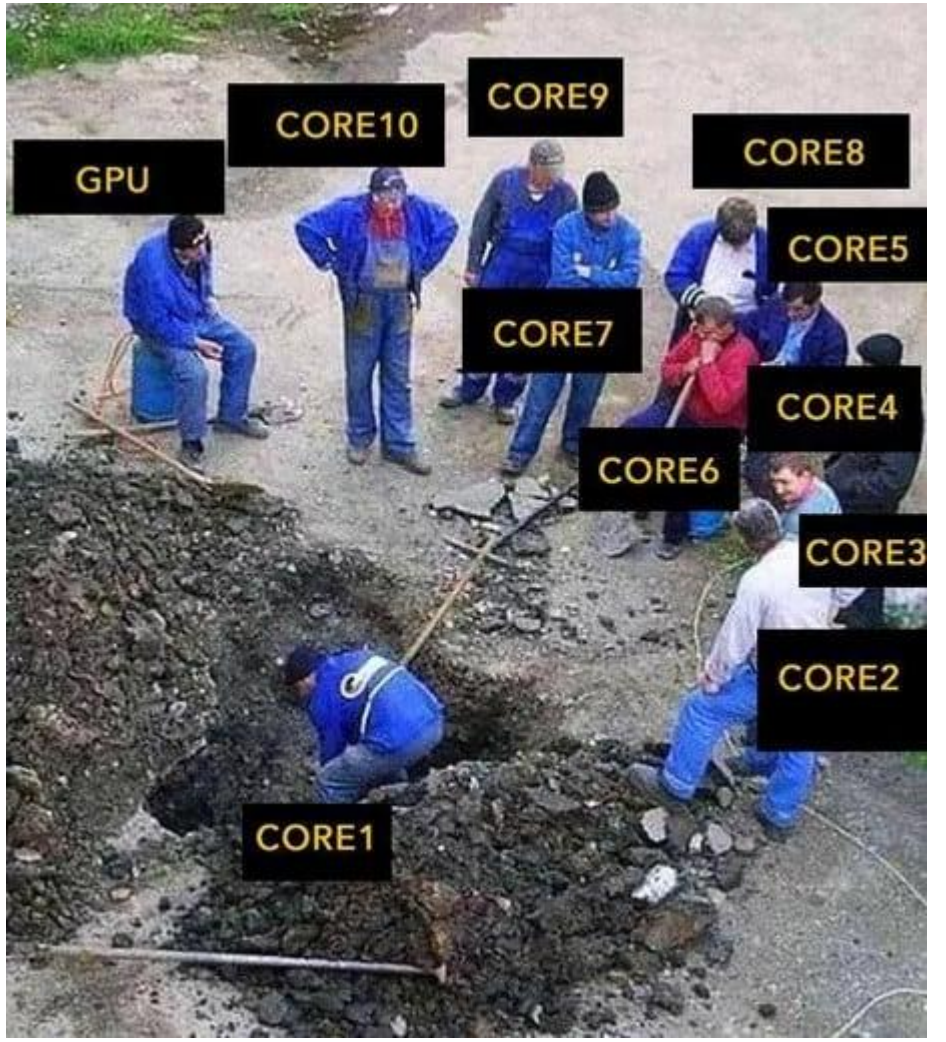
Vehicle Rental

- Take the given classes Vehicle and Person (WDSKI25A-Advanced-Programming/Exercise_5 at main · Marius2311/WDSKI25A-Advanced-Programming) **and build all their interactions** (e.g. a Person should actually rent a car object and so on)
- Try to **integrate either Threading or Multiprocessing into your functions**

- Then, **add a class Building with the subclasses RentalStore and RepairWorkshop**
  - Each should have fitting attributes and methods and should be incorporated into the rental process
  - → When a customer rents and returns a car, he must select the RentalStore
  - → Each returned car will be checked and repaired at a RepairWorkshop (each RepairWorkshop has its own chief mechanic)
  - Come up with own ideas to make the setting realistic

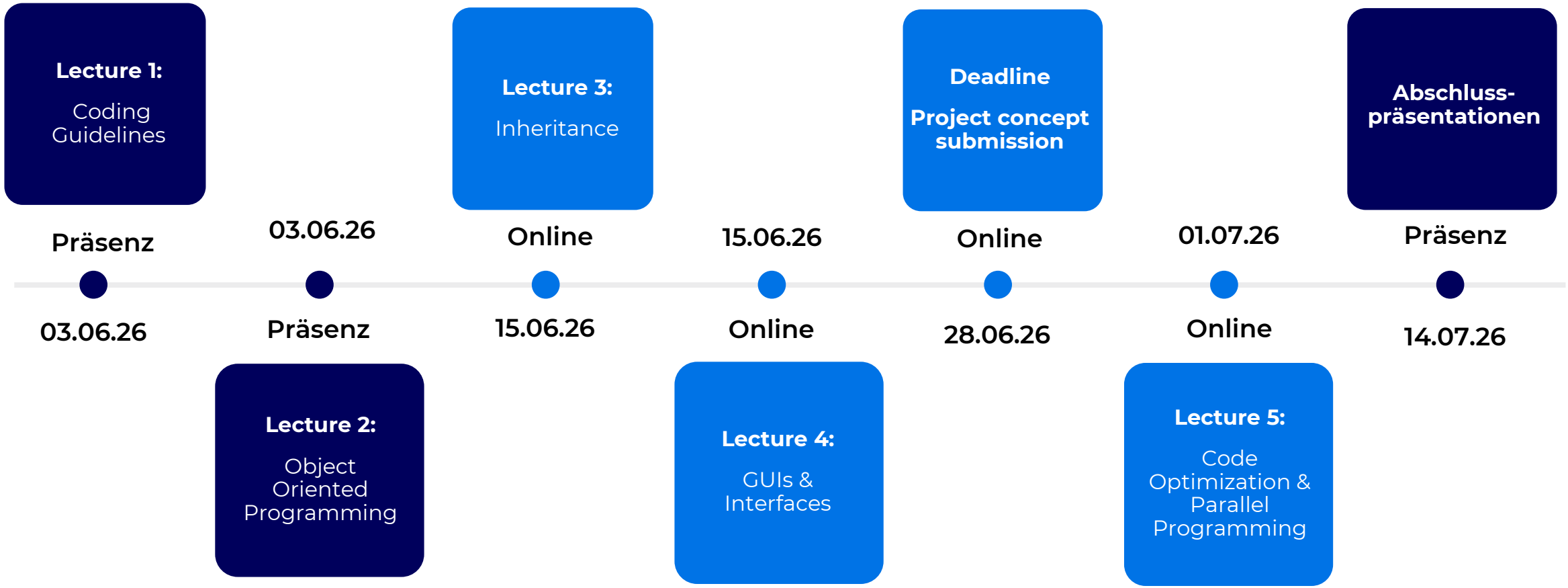
Meme of the day

Threading:



Organisatorisches

Termine



Präsentations-Slots

Abschlusspräsentationen 14.07.2026

Vormittags

- 09:00 – 09:25: Kino-Management
- 09:25 – 09:50: Fitnessstudio
- 09:50 – 10:15: Bruchnormalform
- 10:15 – 10:40: CleanDrive
- 10:40 – 11:05: Tischreservierung
- 11:05 – 11:30: Escape Room
- 11:30 – 11:55: OP-Planung
- 11:55 – 12:20: Habit Builder

Nachmittags

- 13:15 – 13:40: Arztpraxis
- 13:40 – 14:05: ICE-Reservierung
- 14:05 – 14:30: Kalkulationsagent
- 14:30 – 14:55: Energy Grid
- 14:55 – 15:20: Lernquiz
- 15:20 – 15:45: Trockenbau-Termine
- 15:45 – 16:10: Tic Tac Toe
- 16:10 – 16:35: Event-Planner
- 16:35 – 17:00: Bankkonto

Bewertungskriterien

Code-Abgabe

- **Abgabe**:
  - Ein GitHub-Link, der das gesamte Projekt enthält, sodass es sofort ausführbar ist (inkl. requirements.txt und Ausführ-Anweisungen)
  - Eine zip-Datei, die den aktuellen Stand des GitHubs zum Zeitpunkt der Abgabe enthält
  - Die PDF-Datei der Präsentation (entweder als eigenes PDF oder einfach mit im Git)
- **Bewertung:**
  - Commit History wird bewertet! --> Es wird erwartet, dass viele kleine Arbeitspakete (inkl. Branching etc.) committed werden
  - Inhaltliche Richtigkeit
  - Code-Struktur
  - Test-Robustheit
  - Nachvollziehbarkeit
  - Intuitives Ausführen / Installieren

Bewertungskriterien

Präsentation

- Zeitrahmen: 10-20 Minuten + Q&A
- Bewertung:
  - Technik-Sicht
  - Business-Sicht
  - Optik
  - Live-Demo
  - Q&A
  - Präsentationsstil